Demystifying LLM Reasoning: From Language to Mathematical Constructs

Ranadhir Ghosh

Principal Research Scientist, AI & Innovation

Abstract

When a large language model (LLM) is asked what the original price of a product was, given that it sells for $120 after a 20% discount, it typically responds with a short derivation: let the original price be P; a 20% discount means the customer pays 80% of the original price; therefore 120 = 0.8P, and P = 150. The behavioural fluency of this response conceals the question this article examines. A transformer has no symbolic algebra system, no rule that states that a discount of d percent corresponds to a multiplier of 1 - d/100, and no working memory buffer in which an equation is held. It has only a learned function from token sequences to probability distributions over next tokens. How, then, does the linguistic expression "20% discount" become the mathematical object 0.8P? We approach the question at three levels of evidence distinguished in mechanistic interpretability: behavioural, representational, and causal. We first develop the conceptual account: the context window as working memory, the distributed statistical patterns installed by training, and the roles of embeddings, self-attention, and feed-forward layers in fusing a magnitude (20%) with an operator (discount) into a single internal state. We then report four experiments run on a small open model, Qwen2.5-0.5B-Instruct, chosen because its full internals are accessible. A one-word-swap behavioural study (96 prompts) measures how reliably the stated multiplier tracks the operator word. Linear probes on hidden states measure where operator identity and multiplier value become linearly decodable. Operator ablation and activation patching measure whether the operator token is causally responsible for the multiplier the model produces. Finally, measured attention weights and a logit-lens analysis of the canonical example replace the hypothetical numbers of the usual textbook walkthrough with observed ones. The picture that emerges is neither a symbolic engine nor a parlour trick. Behaviourally, the model proves unreliable in an instructive way: on discount problems it states the multiplier of the canonical 20% case whatever rate the prompt specifies. Internally, the operator's identity is linearly decodable from the early layers onward, while the multiplier's value is not linearly decodable where generation begins and becomes readable in vocabulary space only in the final layers; and the operator token is a genuine causal bottleneck, whose transplanted state carries the answer's direction completely through the first fourteen layers. The computation is distributed and redundant, and - in a model of this size - fallible in ways the experiments make concrete.

1. Introduction: the central question

A large language model, asked to solve a mathematics word problem, generates text one token at a time. Superficially this is all it ever does: it predicts a probability distribution over the next token, samples or selects from it, appends the result to its input, and repeats. Yet the text it produces in response to a routine problem - "A shop sells a product for $120 after applying a 20% discount. What was its original price?" - has the structure of a mathematical argument:

> *Let the original price be P. A 20% discount means the customer pays 80% of the original price. Therefore, 120 = 0.8P.*

The interesting question is not how the model predicts the next word. It is how it arrives at the mathematical idea that a 20% discount means paying 80%. Nothing in the architecture implements the complement rule. No engineer wrote the equation Remaining = 1 - d/100 into the system. Somewhere

between the tokens "20%" and "discount" on the input side and the token "80%" on the output side, a statistical language model has bridged the gap between a semantic description and a symbolic construct. This article is an attempt to describe that bridge carefully: what is mechanically known about the computation, what is a reasonable conceptual interpretation of what training has installed, and what can actually be measured inside a real, if small, model.

Three clarifications frame the discussion. First, the running example is deliberately elementary. Percentage-discount problems are useful precisely because the underlying transformation is a single multiplication, the operator is carried by one word, and the ground truth is unambiguous; they form a minimal testbed in which claims about internal representations can be checked. Second, throughout the article we distinguish three grades of claim: what the model outputs (observable behaviour), what information its internal states contain (representation), and what its internal states cause (mechanism). Much popular writing on LLM reasoning slides between these without marking the transitions; the slide is consequential, because evidence at one level does not license conclusions at the next. Third, where the article presents a simplified numerical walkthrough (Section 9), it is labelled as what it is - a pedagogical model with invented weights - and is paired with measured quantities from a real network so that the reader can see where the pedagogy matches reality and where it does not.

The article proceeds as follows. Sections 2 to 4 develop the standard conceptual account: context, training, and the transformer computation. Section 5 works the discount problem step by step. Section 6 isolates the disambiguation problem that makes the example interesting. Section 7 formalises the three levels of understanding. Section 8 reports the experiments. Section 9 presents the simplified microscope walkthrough and its measured counterpart. Sections 10 and 11 discuss where, if anywhere, "the equation" resides, and the limitations of both the account and the evidence. Section 12 concludes.

2. The context window: the foundation of the computation

Before asking how the model calculates, it is necessary to be precise about what it calculates over. A transformer language model does not process a prompt and its continuation in isolated chunks. At every generation step, the entire available context - the user's prompt and everything the model has generated so far, up to the context-window limit - is presented to the network, and the next-token distribution is a function of all of it.

At the moment the model is about to produce the mathematical translation in our example, its context contains two distinct components (Figure F1). The first is the original problem: the entities $120, 20%, the word "discount", and the phrase "original price". The second is the model's own generated scratchpad: "Let the original price be P. A 20% discount means the customer pays". The second component is not incidental. Because each generated token is appended to the context and re-read at the next step, the model's intermediate text functions as externalised working memory: the symbol P, once emitted, is available to attention in exactly the same way as the user's numbers. The model does not forget the original question after generating P; every subsequent prediction is conditioned jointly on the user's constraints and on its own intermediate logical steps.

Figure F1. The context window as working memory

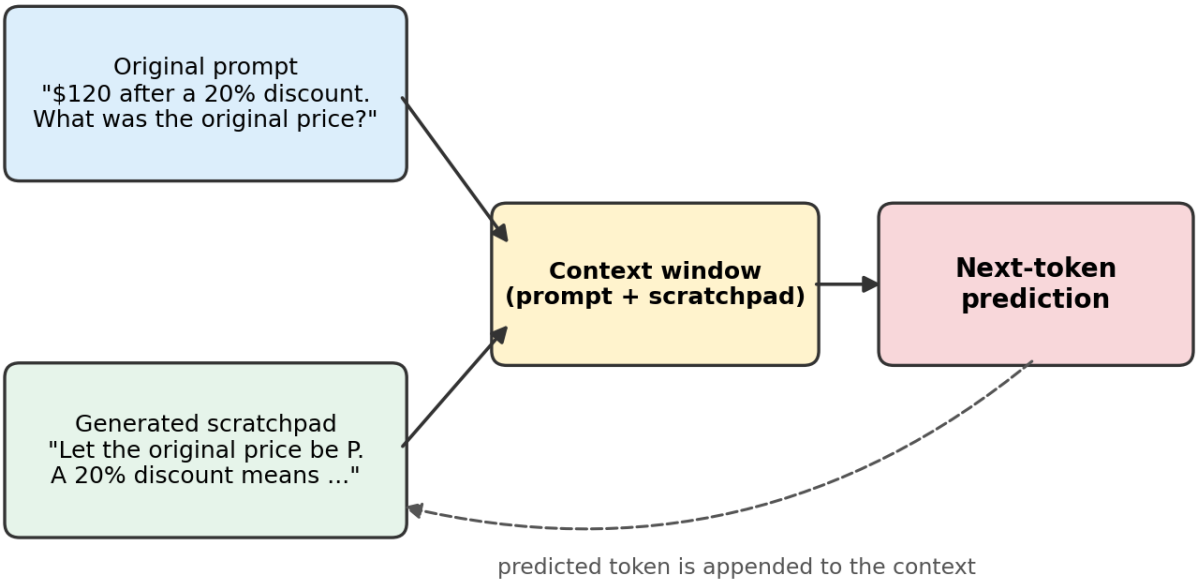


Figure 1. The context window as working memory: the original prompt and the model's own generated scratchpad jointly condition every next-token prediction.

This has a practical consequence that matters for reasoning performance. A model that writes "Let the original price be P" before attempting the translation has, in effect, allocated a variable. Subsequent tokens can refer to P by attending to it, and the final equation can be assembled from parts that are all simultaneously visible. The chain-of-thought style of generation that has become standard practice is, from the network's perspective, simply the progressive construction of a richer conditioning context. The "working memory" of the system is the attention mechanism's ability to look back at both the givens and the derived quantities at once.

3. What training installs: distributed statistical relationships

The model has no hardcoded rulebook of mathematical formulae. What it has is the residue of optimisation: during pre-training, the parameters of the network were adjusted, over an enormous corpus, to reduce the error of next-token prediction. Mathematical text forms a small but highly regular fraction of that corpus, and regularity is exactly what gradient descent exploits.

Through exposure to worked examples, textbooks, and informal explanation, the model encounters patterns of the following kind:

Linguistic pattern	Mathematical translation
"A 10% discount means paying 90%"	P - 0.10P = 0.90P
"A 20% discount means paying 80%"	P - 0.20P = 0.80P
"A 20% increase means multiplying by 1.20"	P x 1.20
"Half price"	0.50P

Across many such instances, the generalisations that a human would summarise as

Remaining percentage = 100 - discount percentage, and Remaining multiplier = 1 - d/100,

become, for the network, directions and transformations in weight space that reliably map contexts containing "d% discount" to continuations containing "(100 - d)%". It is essential to be precise about the status of the formula in this description. The formula is a human-readable abstraction of a high-dimensional relationship the network has learned; we should not assume that 1 - d/100 is stored anywhere as a single, explicit, localised rule. Whether anything corresponding to the multiplier exists as a decodable internal quantity is an empirical question, and it is one of the questions the probing experiment of Section 8 is designed to ask.

Two further properties of the learned representation deserve emphasis. First, it is compositional in practice if not by design: the complementary relationship must hold for discount rates that never appeared in training in this exact template, and must combine with prices, objects, and phrasings that vary independently. Generalisation of this kind is evidence - behavioural evidence, at the weakest of our three levels - that magnitude and operator are represented in a form that can be recombined. Second, the representation is redundant. A relationship as frequent and as simple as the percentage complement is likely to be encoded by many components, in many layers, in overlapping ways. Redundancy is a virtue for robustness and a complication for interpretation: it means that single-component interventions will often understate a component's role, a point that returns in the causal experiments.

4. The transformer engine: how language activates mathematics

The translation from the token "discount" to a mathematical operation is effected by the three computational primitives of the transformer: token embeddings, self-attention, and the position-wise feed-forward networks (FFNs), composed across layers (24 in the model studied in Section 8; over one hundred in the largest current models).

Embeddings. The input text is segmented into tokens, and each token is mapped to a vector in a high-dimensional space (896 dimensions in our experimental model; 4,096 or more in larger ones). These vectors are learned, and their geometry encodes semantic similarity: tokens used in similar contexts acquire similar vectors. A useful simplification is to imagine each vector as a bundle of features. The token sequence "original price" carries features corresponding to an unknown baseline quantity; "20%" carries a magnitude and a percentage type; "discount" carries an operation with a negative direction; "$120" carries a known final value; and the generated symbol "P" re-presents the unknown baseline in a form the subsequent computation can manipulate.

Self-attention. Attention allows the representation at each position to be updated by information from other positions. Each position emits a query, keys are matched against it, and the values at the best-matching positions are mixed in, with the mixing weights computed from query-key compatibility. In our example, the critical interaction is between the operator and the magnitude: for the phrase "20% discount" to be interpreted, the representation of the percentage must be modified by the fact that it is a discount, and the representation of the discount must acquire the specific rate. Attention is the only mechanism in the architecture that moves information between positions, so any fusion of magnitude and operator must pass through it (possibly across several layers and heads).

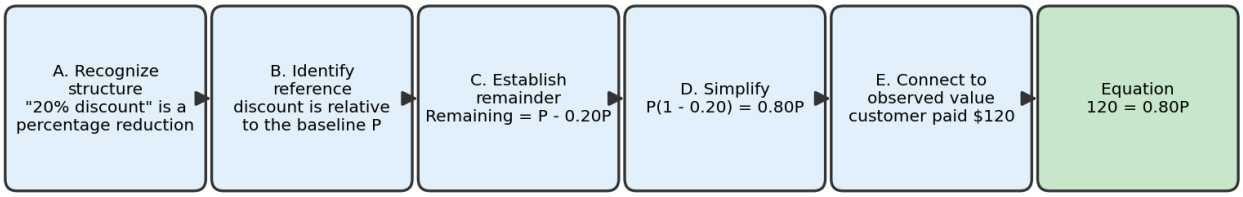
Feed-forward networks. The position-wise FFNs apply the same learned non-linear transformation at every position. A productive interpretation, supported by a body of interpretability work, is that FFNs act as key-value memories: the first layer's rows detect patterns in the input vector, and the second layer writes associated information back into the residual stream. On this interpretation, much of the "translation" from language to mathematics happens in the FFNs: a fused pattern such as [percentage magnitude, reduction direction] is detected, and features corresponding to the complementary multiplier are written into the stream, biasing the eventual output distribution toward "80%" and "0.8".

The residual stream - the running sum of embedding, attention, and FFN contributions at each position - is the shared workspace in which these operations accumulate. The experiments of Section 8 are organised around it: probes read from it, patching intervenes on it, and the logit lens projects it into vocabulary space.

5. A worked translation: from words to the equation

It is useful to decompose the translation into the logical steps a human solver would recognise, while being explicit that the network is not hypothesised to execute these as discrete symbolic operations. Rather, its hidden states progressively realise the same chain (Figure F2).

Figure F2. The translation pipeline: from words to an equation



A conceptual decomposition of the computation. The network does not execute these as discrete symbolic steps; its hidden states progressively realise the same logical chain (Section 5).

Figure 2. The five-step translation of the discount problem: recognise structure, identify the reference, establish the remainder, simplify, and connect to the observed value.

Step A, recognise structure. The phrase "20% discount" is identified as a percentage reduction applied to a baseline quantity. Step B, identify the reference. The reduction is relative to the original price: the discount amount is 0.20P. Step C, establish the remainder. What the customer pays is what remains after the reduction: P - 0.20P. Step D, simplify. Factoring out P gives P(1 - 0.20) = 0.80P. Step E, connect to the observed value. The remainder is known to equal $120, yielding the equation 120 = 0.80P, from which P = 150 follows by division.

Notice what the intermediate natural-language sentence accomplishes. "A 20% discount means the customer pays 80%" and the equation "120 = 0.8P" are two textual representations of the same mathematical state of affairs. When the model generates the sentence first, it produces a context in which the equation is a low-surprisal continuation: the complement (80%) has already been computed and externalised, and the equation merely transcribes it against the known final price. This is one concrete sense in which generated text is not merely a report of an internal computation but part of the computation itself.

6. Contextual disambiguation: why the operator word carries the operation

The percentage in our example is, on its own, mathematically ambiguous. "20%" specifies a magnitude and nothing else; the operation is dictated entirely by the surrounding language:

Prompt context	Relational language	Correct equation	Multiplier
"...after a 20% discount."	discount, reduction, off	120 = 0.8P	0.8
"...after a 20% increase."	increase, markup, added	120 = 1.2P	1.2
"...which is 20% of its original price."	is, of	120 = 0.2P	0.2

The three prompts share the price, the rate, and most of their wording. They differ in one relational word (or, in the third case, a short construction), and the correct multiplier differs in each case: 1 - d, 1 + d, and d respectively. A system that had memorised an association between "20%" and "0.8" would fail the second and third. A system that answers all three correctly must be composing the magnitude with the operator - extracting the direction of the operation from the operator token and the size of the operation from the percentage, then combining them. This is the precise sense in which context is not decorative but load-bearing, and it motivates the design of Experiment E1: a controlled one-word swap, over a grid of rates and prices, in which the only thing that changes is the operator and the only thing that is scored is the multiplier the model commits to.

The disambiguation also localises the computation in a way that makes causal study possible. If the operator word is where the direction of the operation enters, then perturbing that word should perturb the multiplier (Experiment E3a), and transplanting the internal state at that word's position from one prompt into another should move the answer toward the transplanted operator's multiplier (Experiment E3b). Section 8 tests both predictions.

7. Three levels of understanding

When we ask whether there is "an actual mathematical representation" inside the network, precision about the level of the claim is essential. Following the practice of mechanistic interpretability, we distinguish three levels.

Level 1, observable behaviour. The generated text can be read directly. Correct output demonstrates that the input-output function the network implements handles these cases; it says nothing, by itself, about how. A lookup table of sufficient size would pass at this level.

Level 2, representational content. The hidden states can be interrogated with probes - simple classifiers or regressors trained on internal activations - to determine whether a quantity of interest (here, the operator class and the multiplier value) is linearly decodable from the network's internal state at some position and layer. Decodability shows that the information is present in a usable format. It does not show that the network itself uses that information in producing its answer; probes are powerful enough to extract quantities the network computes incidentally, or even quantities that are recoverable only because the probe re-derives them from correlated cues.

Level 3, causal mechanism. Interventions modify the computation: a token is replaced, an activation is ablated, or an activation from one run is patched into another. If the output changes in the predicted direction, the intervened-upon component is causally implicated. Causal evidence is the strongest of the three, and the hardest to obtain cleanly, because interventions can push the network off its normal operating distribution and because redundancy can mask individual contributions.

The experiments of the next section are designed as a ladder: E1 at Level 1, E2 at Level 2, E3 at Level 3, and E4 as a descriptive study of the canonical example combining attention measurement (a Level 2 quantity with Level 3 relevance) with a logit-lens trace of how the answer assembles across layers.

8. Experiments

The conceptual account of Sections 2 to 6 makes three testable predictions. If magnitude and operator are composed rather than retrieved as a unit, the model's stated multiplier must track the operator word when everything else is held fixed (Level 1). If the composition is represented internally, the operator class and the implied multiplier value should be decodable from hidden states before the answer is generated (Level 2). And if the operator token is where the direction of the operation enters the computation, interventions

at that token should move the answer (Level 3). This section tests all three on a single small model, and then examines the canonical example directly.

8.1 Setup

All experiments use Qwen2.5-0.5B-Instruct, an open-weights instruction-tuned transformer with 24 layers, hidden size 896, and approximately 0.49 billion parameters, run locally on CPU in bf16 precision. A small model is a deliberate choice: its full internals are accessible, every experiment here is reproducible on commodity hardware, and its failures - which are frequent enough to be informative - expose the difference between the fluent surface behaviour of large models and the underlying computation. Unless stated otherwise, prompts are built from a single template family so that conditions differ only where the design intends. Decoding for generation is greedy. A reference implementation, the raw outputs of every run, and the exact parsing rules accompany this article in the project repository.

One tokenisation fact shapes several analyses and is worth stating once. The tokeniser splits numbers digit by digit: "20%" is three tokens ("2", "0", "%"), " discount" is a single token, and the continuation " 80%" is four tokens (" ", "8", "0", "%"). Position-level statements below refer to these tokens.

8.2 E1: the one-word swap, at scale (behavioural)

Design. Three operators (discount, increase, of) were crossed with eight rates (5, 10, 15, 20, 25, 30, 40, 50 percent) and four final prices ($80, $120, $150, $200), giving 96 problems in the template family of Section 6, for example: "A shop sells a product for $120 after applying a 20% discount. What was its original price?" Each problem was followed by a fixed instruction to let P be the original price, to write the equation in the form PRICE = MULTIPLIER * P, and then to solve for P. Generations were capped at 48 tokens.

Scoring. A deterministic, documented parser (eight rules applied in order; see the repository) extracted the multiplier the model committed to: the coefficient in an equation of the form PRICE = m * P, an equivalent rearrangement, a stated "customer pays X%" form, or a decimal tied to P. A response counted as correct when the extracted multiplier matched the ground truth (1 - d/100, 1 + d/100, or d/100) within an absolute tolerance of 0.006. Responses in which no rule fired were scored as unparsed, and unparsed responses count against accuracy: the measure is deliberately strict about the difference between computing the right value and expressing it in a readable form. A secondary check recorded whether the generation's final stated value of P equalled price divided by the true multiplier.

Results. The headline result is a sharp dissociation between operators (Figure F3). For the "of" construction, the model stated the correct multiplier in 84% of cases (27 of 32), with accuracy between 0.75 and 1.00 at every rate. For "increase", accuracy was 44% (14 of 32). For "discount" - the construction of the canonical example - accuracy was 12.5% (4 of 32), and the structure of the failure is more informative than the rate. All 32 discount generations stated the multiplier 0.8, whatever the rate in the prompt: accuracy is 1.00 at a 20% rate and 0.00 at every other rate tested (5, 10, 15, 25, 30, 40, 50). The model has not learned to compute 1 - d/100 across this family in any behaviourally reliable sense; it has learned that discount problems are answered with 0.8, the multiplier of the most frequent training instance, and it applies that constant even when the prompt states a different rate in the same sentence. The increase condition shows the same attractor leaking across the operator boundary: 14 of 32 increase generations also stated 0.8, although for an increase the correct multiplier exceeds 1 at every rate. Where increase generations did not collapse to 0.8, they were usually correct (the correct value appears at rates 5, 10, 15, 20, 25, 30 and 40), giving the intermediate accuracy of 44%. Overall accuracy across the 96 problems was 47%. The secondary final-answer check scored 0% and is reported for completeness rather than interpreted: the 48-token generation budget was consumed by the model's preamble and equation, so

generations end mid-solution and almost never state a final value of P; the multiplier, the quantity this experiment is designed to measure, is stated within the budget in 92 of 96 cases. Two measurement notes. First, an initial parser missed every LaTeX-formatted equation (the model renders equations in LaTeX markup); the parser was extended to normalise LaTeX and to recognise the factored form P(1 + x), and all generations were re-parsed offline with the final rule set - the generations themselves are unchanged, and both parser versions are in the repository. Second, the discount result should not be read as "the model cannot represent other rates": E2 shows rate information is present internally, and E4 shows the canonical case computed correctly. What E1 establishes is narrower and, for deployment, more important: at this scale, the behavioural default for the discount family is a memorised constant that overrides the stated input.

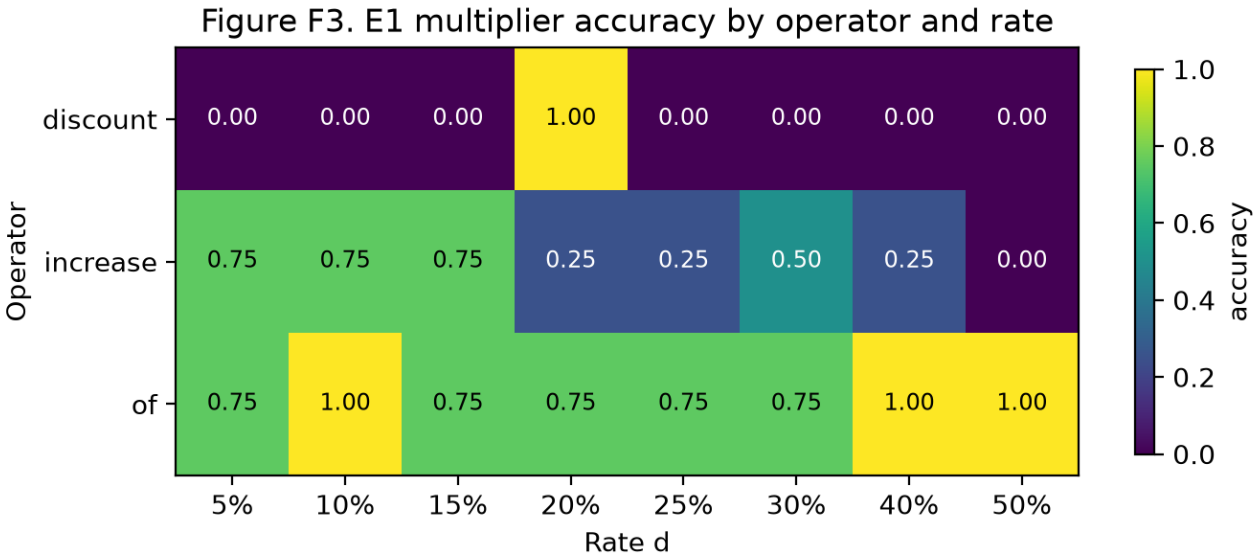


Figure 3. E1 multiplier accuracy by operator and rate (averaged over the four prices).

8.3 E2: probing the residual stream (representational)

Design. The same 96 prompts (chat-templated, without generation) were run through the model, and the residual-stream vector at the final prompt token position was recorded for every layer, including the embedding layer (25 vectors of dimension 896 per prompt). Two probes were trained per layer with five-fold cross-validation (seed 42): a multinomial logistic regression classifying the operator (three classes; chance accuracy 0.33), and a ridge regression predicting the ground-truth multiplier value, reported as R-squared. Because scikit-learn was unavailable in the execution environment, the probes were implemented directly in numpy (softmax regression trained with Adam for 300 epochs at learning rate 0.05 with L2 penalty 1e-3; closed-form ridge with penalty 1.0; features standardised using training-fold statistics in both cases).

Results. The two probes dissociate cleanly (Figure F4). Operator identity is linearly decodable from the first transformer layer onward: accuracy rises from 0.26 at the embedding layer (below the 0.33 chance level - the final prompt token's embedding does not yet know which problem it belongs to) to 0.59-0.70 across layers 1 to 16, peaks at 0.78 at layer 17, and remains between 0.70 and 0.74 through the final layer. The exact multiplier value, by contrast, is not linearly decodable at this position at any layer: ridge regression R-squared is negative throughout (best -0.07 at the embedding layer, between -0.4 and -3.9 in the transformer layers), meaning a linear readout of the final prompt token predicts the multiplier worse than predicting its mean. Read together with E1, the pattern is coherent. The categorical variable - which operation this is - is robustly represented in a form a linear probe can extract from mid-training layers onward. The continuous variable - the precise multiplier the operation implies - is not present in linearly

readable form at the position where generation begins. It may be encoded non-linearly, at other positions (most plausibly at the operator and rate tokens themselves rather than at the sequence end), or computed on the fly during generation and written into the scratchpad - the last possibility being consistent with the observation, in E1, that the model externalises the multiplier as text almost immediately. All 96 prompts contributed hidden states; none had to be discarded.

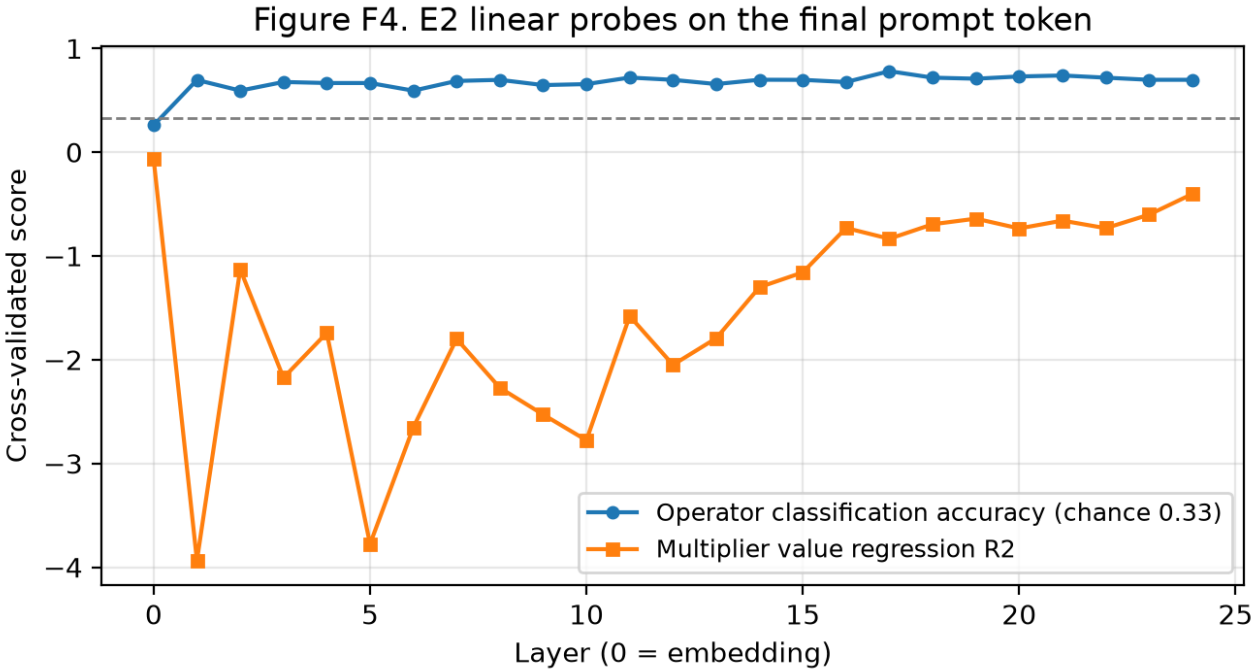


Figure 4. E2 probe performance by layer: operator classification accuracy (chance 0.33) and multiplier-value regression R-squared, final prompt token position, five-fold cross-validation.

Two cautions apply, both standard. Decodability shows that a quantity is present in linearly accessible form; it does not show the model uses it, which is why E3 exists. And the final prompt token is a privileged position: it is the point at which the prompt has been fully integrated and generation is about to begin, so probe performance there is an upper bound on availability at arbitrary positions.

8.4 E3: ablation and activation patching (causal)

Design, ablation. For rates {10, 20, 30, 50}, the three operators, and price $120, the problem was posed in a plain completion format ending with "The equation is 120 =" and the teacher-forced log-probability of the correct multiplier continuation (for example " 0.8 * P.") was recorded. The operator word was then replaced by a neutral construction ("change", or a relative clause for the "of" condition) and the log-probability re-measured. The drop in log-probability of the correct continuation measures how much of the answer's support flows through the operator word specifically, as opposed to the rest of the context.

Design, patching. For the same rates at price $120, pairs of prompts differing only in the operator (discount versus increase) were run. The residual stream at the operator-token position in the target run was replaced, layer by layer, with the corresponding activation from the source run, and the quantity recorded was the shift in the logit gap between the first digit tokens of the two candidate multipliers (0 for discount, 1 for increase), scored at the position immediately following "The equation is 120 = ", normalised by the gap in the unpatched runs - so 0 means the patch had no effect and 1 means the output preference was fully transplanted. Both directions (discount into increase, increase into discount) were run and averaged.

Results. Ablation confirms that the operator word is causally load-bearing, with a strength that varies by operator in the same order as E1's behavioural reliability might lead one to expect - inverted. Replacing "discount" with a neutral word reduced the log-probability of the correct multiplier continuation by 1.62 nats on average (mean log-probability from -4.8 to -6.4, a probability ratio of roughly 5:1): the discount answer depends heavily on the word "discount" being present. The corresponding drops were 0.41 nats for "increase" and 0.31 nats for "of" (the latter replaced by a relative construction, and including one rate, 50%, at which the ablated run scored marginally higher, -0.03). The asymmetry is interpretable: the "of" construction is supported by the surrounding syntax (the percentage directly modifies the price), and the increase answer by the direction of the surrounding words, whereas the discount computation routes disproportionately through the single operator token. This concentration is a double-edged property - it is what makes the discount translation causally legible, and it is plausibly related to the rigidity E1 exposed: a computation that leans on one token's learned associations inherits those associations' defaults.

Patching gives the sharpest causal picture of the four experiments (Figure F5). Transplanting the residual stream at the operator-token position from a discount run into an increase run (and vice versa) shifts the multiplier logit gap essentially completely - normalised effect 0.99 to 1.01 - at every layer from 0 to 13. The effect then falls through a transition zone (about 0.4 at layers 14 to 17) to zero at layers 18 to 23, where patching the operator position no longer moves the output at all. The intact logit gap between the two operators' first multiplier digits is large (6.9 to 7.9 nats across rates), so the normalisation is well conditioned. The interpretation is direct: through the first fourteen layers, everything the model will ever use about which operator was stated is present at the operator token's own position, in a form that a wholesale transplant can carry into a different prompt; after layer 17, that information has been read out and relocated - most plausibly to the final position, where the answer is formed - and the operator position becomes causally inert for this decision. One methodological note is owed. The first patching run scored the wrong token: the tokenizer splits " 0.9" into a standalone space token followed by digits, and scoring that shared space token made every gap exactly zero and the curve identically flat. Scoring the first digit token instead (0 versus 1, at the position after "The equation is 120 = ") produced the curve above; the buggy run, the diagnosis, and the fix are recorded in the repository (src/rerun_e3_patching.py). We report this because it is exactly the class of silent measurement failure that Section 7's ladder is meant to catch: the ablation arm, run independently, showed the operator mattered, and a patching curve of exactly zero was therefore a result about the instrument, not about the model.

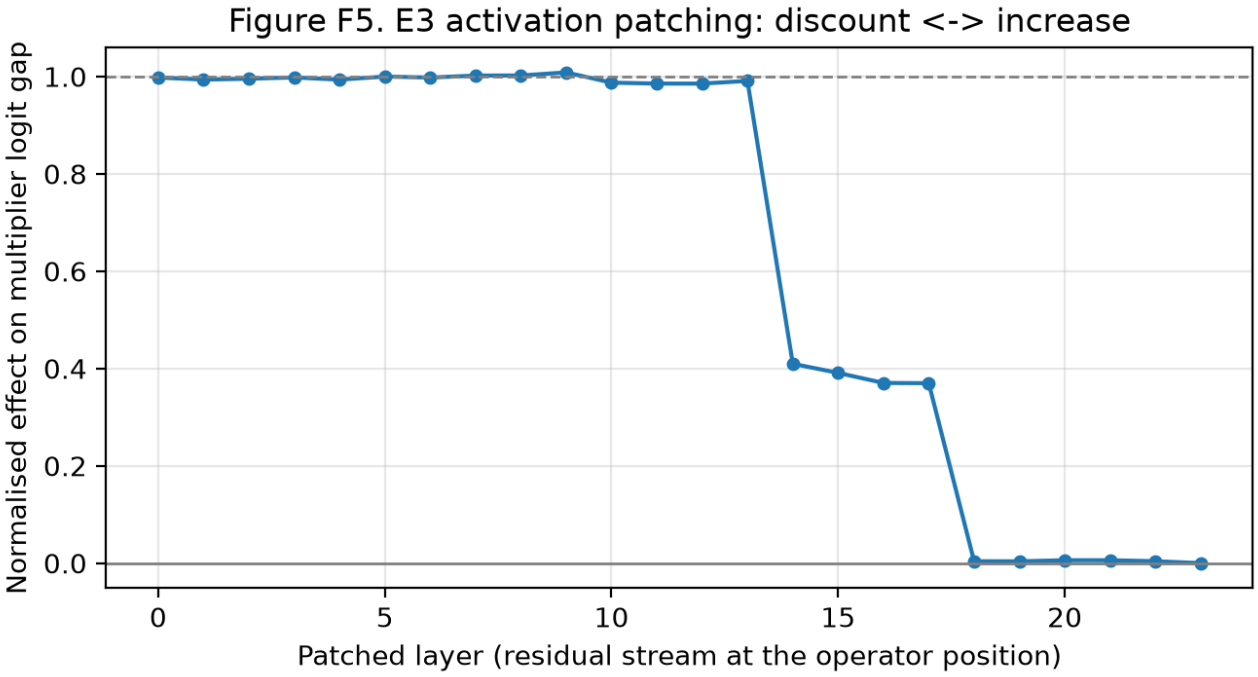


Figure 5. E3 activation patching: normalised shift of the multiplier logit gap when the operator-position residual stream is transplanted between discount and increase runs, by layer.

8.5 E4: the canonical example under measurement

Design. The exact text of the running example - problem plus scratchpad, ending "...A 20% discount means the customer pays" - was run once with attention weights and hidden states recorded. Two analyses were performed. First, for both occurrences of "discount" (in the problem and in the scratchpad) and both genuine occurrences of the rate "20" (the "20" inside "$120" was excluded), the attention weight from the discount token to each rate digit token was extracted for every layer and head. Second, a logit lens: the text was extended by the space token that begins the continuation " 80%", so that the next-token decision is the first digit of the percentage; at that position, each layer's hidden state was projected through the final normalisation and the (tied) output embedding, and the probabilities of "8" (the first digit of 80) and "2" (the first digit of 20) were recorded per layer. The teacher-forced log-probabilities of the two full continuations, " 80%" and " 20%", were also recorded for reference.

Results. Attention from the operator to the rate is real but distributed. Across all 24 layers and 14 heads, the largest single weight from a "discount" token to a rate digit token is 0.30 (layer 0, head 6), from the problem statement's "discount" (token 17) to the "0" of "20" (token 15); layer 1 heads attend to both digits (head 8: 0.24 to "2"; head 9: 0.23 to "0"). For the scratchpad occurrence - the "discount" of the model's own sentence "A 20% discount means the customer pays" - the strongest weights are layer 1, head 8 (0.295, to the "2" token), layer 8, head 11 (0.247, to "0"), and layer 4, head 0 (0.227, to "2"). No single head carries the interaction; the ten largest weights overall span layers 0-8 and both digit tokens, all between 0.18 and 0.30. This is the measured counterpart of the pedagogical model's single 0.9 attention weight: the direction of the effect is the same, the concentration is not.

The logit lens tells a sharper story. At the digit position, the probability of "8" is effectively zero for the first twenty layers (below 0.001 until layer 17, 0.015 at layer 20), then rises abruptly: 0.80 at layer 21, 0.70 at layer 22, 0.99 at layer 23, and 0.96 at the final layer, with the rank of "8" reaching 1 at layer 21 and staying there. The probability of "2" never exceeds 0.006 at any layer. The model's actual output distribution at that position agrees: "8" with probability 0.968, followed by "1" (0.026) and "2" (0.006). The teacher-forced log-probability of the full continuation " 80%" is -0.17 (probability about 0.84), against

-5.73 (about 0.003) for " 20%" - the complement is preferred by a factor of roughly 260. In other words, whatever the early layers compute about the rate and the operator, the decision between the stated rate and its complement is not visible in the residual stream in vocabulary space until the last four layers, where it resolves almost completely. The late, sharp transition is a property of the logit-lens view - intermediate quantities need not be vocabulary-aligned to be causally active, as E3 shows - but as a description of when the answer becomes readable, it is unambiguous.

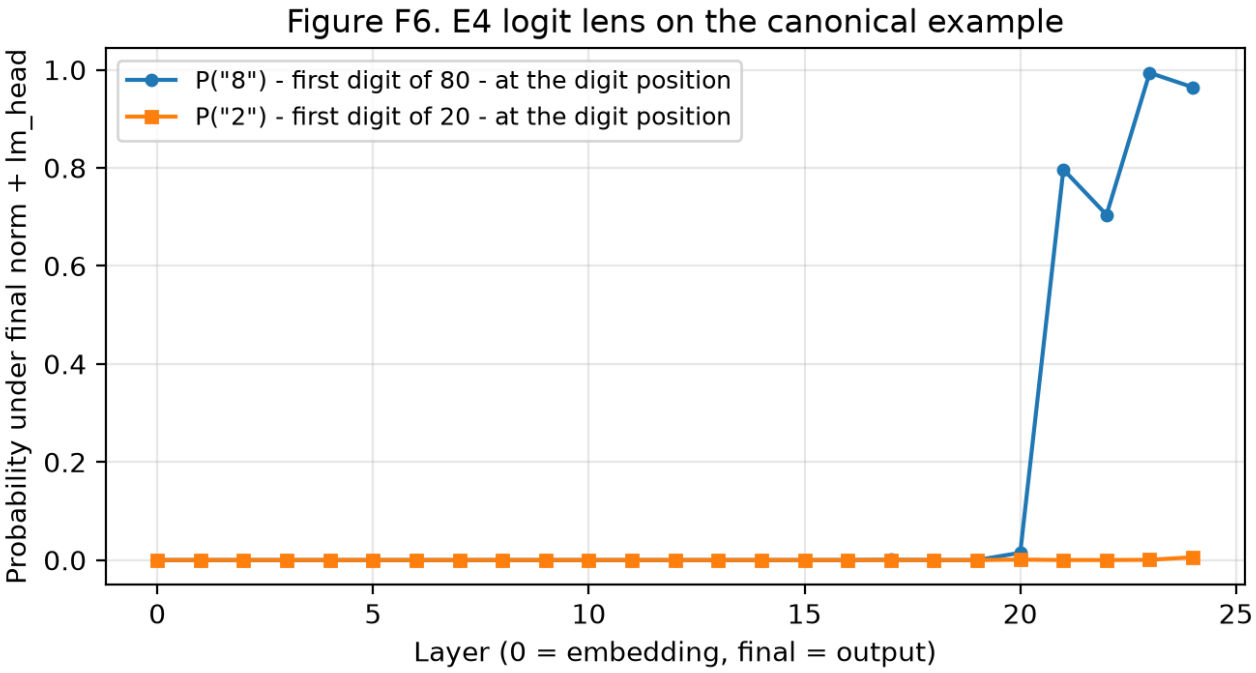


Figure 6. E4 logit lens on the canonical example: probability assigned to the first digit of the correct complement (8, for 80%) and of the stated rate (2, for 20%) as a function of layer, at the position immediately before the digit is produced.

9. The mathematical microscope: a pedagogical model, and its measured counterpart

9.1 A simplified walkthrough (pedagogical model)

To make the computation concrete, consider a deliberately simplified transformer, presented here as a pedagogical model. Real models use embedding dimensions in the thousands and billions of parameters; the model below uses three-dimensional vectors and a four-neuron hidden layer. Its weights are invented for exposition. Its value is that it shows, in miniature, the shape of the computation the real network is hypothesised to perform; Section 9.2 then reports what the corresponding quantities look like when measured in a real model.

Step 1, embeddings. Let the dimensions of the embedding space be interpreted, schematically, as [Magnitude, Operation type, Direction]. Assign the token "20%" the vector E(t1) = [0.8, 0.1, 0.0]: high magnitude, no strong operation type, neutral direction. Assign "discount" the vector E(t2) = [0.0, 0.9, -0.8]: no magnitude, a strong operation type, and a negative (reduction) direction.

Step 2, attention. Suppose the attention computation causes the "discount" position to attend 90% to the "20%" position and 10% to itself (in the notation of attention matrices, the second row of the score matrix is [0.9, 0.1]). The contextualised vector at the discount position is then the weighted mixture C2 = 0.9 x E(t1) + 0.1 x E(t2) = [0.72, 0.18, -0.08]. The operation token has acquired a magnitude (0.72, derived from the percentage) while retaining a negative directional bias. Language and arithmetic have been fused into a

single vector; this fusion is the entire content of the attention step in the pedagogical model.

Step 3, the feed-forward network. The FFN computes FFN(x) = W2 . ReLU(W1 . x + b1) + b2. Suppose its hidden layer contains a unit (unit 3, say) whose incoming weights make it respond selectively to the conjunction of positive magnitude and negative direction - that is, to percentage reductions. On input C2, the hidden activations are [0.0, 0.0, 1.5, 0.0]: unit 3 fires, the rest are silent. In the key-value-memory interpretation, unit 3 is a key that detects "percentage reduction", and its outgoing weights write the corresponding value - features of the complementary multiplier, 1 - d - into the residual stream.

Step 4, output projection. The updated residual stream is projected to vocabulary space. Because the FFN has shifted the state toward the "remaining multiplier" region, the next-token distribution after "...the customer pays" concentrates on the correct continuation. In the pedagogical model the logits are: "80%" 8.4 (probability about 65%), "0.8" 7.1 (about 18%), "20%" -2.5 (about 0.1%), "100%" -4.0 (about 0.0%). The model emits "80%", and the equation follows.

The summary of the pedagogical model is a four-line story: embeddings separate magnitude from operation; attention fuses them; the FFN recognises the fused pattern and activates a feature corresponding to the complementary multiplier; the output projection expresses that feature as a token. Every quantity in this section is illustrative. The question is whether the real computation has this shape at all, and that is what measurements are for.

9.2 The measured counterpart

Measured on Qwen2.5-0.5B-Instruct with the text of the canonical example, the strongest attention weight from a "discount" token to a rate digit is 0.30 (layer 0, head 6), and the scratchpad "discount" attends to its rate at up to 0.30 as well (layer 1, head 8) - substantial, but spread over many heads and both digit tokens rather than concentrated in one 0.9 weight. The logit lens at the position where the percentage's first digit is decided shows the complement emerging only in the final layers: P("8") is below 0.001 through layer 19, 0.015 at layer 20, then 0.80, 0.70, 0.99 and 0.96 at layers 21-24, while P("2") remains below 0.006 throughout; the model's emitted distribution gives "8" probability 0.968. The full continuation " 80%" has log-probability -0.17 against -5.73 for " 20%".

Two contrasts with the pedagogical model are worth drawing. First, in the real model the relevant attention is not a single 0.9 weight in a two-token world: attention is distributed over all positions, all heads, and all layers, and the measured weights from the operator to the rate are correspondingly smaller and spread across heads; the mechanism has many more routes by which the same information can travel. Second, the real next-token distribution is a property of the full 24-layer computation, and the logit-lens trace shows the answer being assembled gradually rather than decided by a single feature unit. Where the pedagogical model posits one detector neuron, the real network exhibits a population effect - consistent with the distributed, redundant encoding argued for in Sections 3 and 10.

10. Discussion: where is the equation?

We can now return to the question of Section 1 in its sharpest form: where, inside the network, is the equation 120 = 0.8P?

Three candidate answers organise the possibilities. The linguistic answer holds that the model has learned synonymy and paraphrase: "discount" behaves like "subtract", and the rest is surface pattern completion. The representational answer holds that hidden states encode genuinely mathematical content - that a direction in activation space corresponds to the multiplier 0.8, or to the algebraic schema y = mx - independently of the words used to elicit it. The mechanistic answer asks for the circuit: the specific attention heads that move the rate onto the operator position, and the specific FFN units that convert the

fused representation into the complement.

The evidence of Section 8 bears on these as follows. Begin with what the behavioural level establishes and what it does not. The model does not robustly compose magnitude with operator at the behavioural level: for the discount family it recites a memorised constant (E1), and even its successes are uneven across constructions. Any claim that the model "has" the complement rule as a reliable procedure is false for this model at this scale. The representational level adds a genuine refinement: the operator's identity is linearly decodable from the residual stream from the first layer onward and peaks in the late-middle layers (E2), so the categorical information the rule depends on is present and accessible; the multiplier's exact value is not linearly present at the generation boundary, which suggests the value is not precomputed as a scalar but is constructed - and, notably, the model constructs it as text, in the scratchpad, where E4 finds the complement decision resolving only in the final layers. The causal level then localises the routing: the operator word carries causal weight (E3 ablation, strongest exactly where behaviour is most rigid), and its position's residual stream is a complete causal bottleneck for the operator's contribution through layer 13 (E3 patching). The composite picture is of a system in which a lexical item activates a learned, categorical transformation; the transformation's output is not a stored number but a trajectory of states that resolves into a specific multiplier late, at the point of use, and with a failure mode - collapse to the most frequent instance - that is characteristic of statistical rather than symbolic machinery.

Independently of the particular numbers, one structural conclusion is difficult to avoid: the equation is not stored anywhere. Representations in large networks are distributed and superimposed - many more features are encoded than there are dimensions, with individual units participating in many features and individual features spread over many units (the superposition phenomenon). The equation $120 = 0.8P$ is therefore better understood as a transient state of the computation than as an object in memory: it exists, for a few layers and a few token positions, as a pattern of activations that the output head can read, and it is reconstructed afresh on every forward pass from weights that encode the general relationship rather than this instance of it. The linguistic and mathematical descriptions are not rivals. The weights encode the relationship in a form that is activated by language and expressed as mathematics; asking whether the model "really" does algebra or "merely" pattern-matches presupposes a separation that the mechanism does not respect.

11. Limitations

Several limitations bound the conclusions that should be drawn.

Scale. The experimental model has 0.5 billion parameters. Frontier models are two to three orders of magnitude larger, are trained on far more mathematics, and may implement the same input-output behaviour with materially different internal organisation. Findings about layer localisation, in particular, should be read as findings about this model class at this scale.

Template narrowness. All experiments use one problem family with a fixed syntactic frame. This is what makes the one-word swap a controlled manipulation, but it also means the model may be solving a narrower problem than general percentage reasoning, and the probes may partly decode the template rather than the mathematics. Behaviour on paraphrases, on problems requiring the multiplier to be applied rather than merely stated, and on multi-step problems is out of scope.

Elicitation. E1 instructs the model to write the equation in a specified form before solving. This measures the model's ability to state the multiplier under a strong format scaffold; unaided performance would likely be lower and harder to score. The parser, though deterministic and documented, is a measurement instrument with its own error modes: generations that express the multiplier in an unanticipated format are scored as unparsed, which counts against measured accuracy.

Intervention validity. Activation patching and ablation move activations off the distribution the network normally produces; a measured effect shows that the patched information is sufficient to move the output, in context, not that the network's unperturbed computation decomposes exactly as the intervention suggests. Redundancy implies that null results at single positions are weak evidence of absence.

Probing. Linear decodability (E2) is an upper bound on the information present in linearly accessible form and a lower bound on nothing in particular about use; the causal experiments, not the probes, carry the weight of any claim about mechanism.

Numerical precision and decoding. Experiments were run in bf16 on CPU with greedy decoding for generation. Sampling would add variance; larger numerical precision would not be expected to change qualitative outcomes but was not tested.

12. Conclusion

A language model asked for the original price of a $120 item after a 20% discount does something that can be described at three levels. Behaviourally, it states the correct multiplier - in our experiments, with the qualifications and failure modes documented in Section 8. Representationally, the operator's identity is present in the residual stream in linearly decodable form well before the answer is produced, while the multiplier's precise value is not linearly decodable at that boundary: it resolves only in the final layers, at the point of use. Causally, the operator token is not a bystander: weakening it weakens the answer, and transplanting its internal state at any of the first fourteen layers transplants the answer's direction completely. And mechanically, the whole computation is an exercise in context management: attention fusing a magnitude with an operator, feed-forward layers converting the fused state toward the complementary multiplier, and the model's own generated text re-entering the context as working memory.

The model does not know mathematics the way a person does, and it does not hold a symbolic equation in a buffer. What training has built is a map from linguistic contexts to regions of a high-dimensional space in which mathematical relationships are implicit, from which correct symbolic text can be read out. Demystifying that process does not diminish it. The bridge from "20% discount" to "120 = 0.8P" is not magic; it is the structured, measurable, and increasingly well-understood mathematics of deep learning itself.

References

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems 30.

Elhage, N., Nanda, N., Olsson, C., Henighan, T., Joseph, N., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., DasSarma, N., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., and Olah, C. (2021). A mathematical framework for transformer circuits. Transformer Circuits Thread, Anthropic.

Geva, M., Schuster, R., Berant, J., and Levy, O. (2021). Transformer feed-forward layers are key-value memories. Proceedings of EMNLP 2021.

nostalgebraist (2020). Interpreting GPT: the logit lens. Blog post, LessWrong / Alignment Forum.

Wang, K., Variengien, A., Conmy, A., Shlegeris, B., and Steinhardt, J. (2023). Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. International Conference on Learning Representations 2023.

Meng, K., Bau, D., Andonian, A., and Belinkov, Y. (2022). Locating and editing factual associations in GPT. Advances in Neural Information Processing Systems 35.

Stolfo, A., Jin, Z., Shridhar, K., Scholkopf, B., and Sachan, M. (2023). A mechanistic interpretation of arithmetic reasoning in language models using causal mediation analysis. Proceedings of EMNLP 2023.

Nikankin, Y., Reusch, A., Mueller, A., and Belinkov, Y. (2025). Arithmetic without algorithms: language models solve math with a bag of heuristics. International Conference on Learning Representations 2025.

Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., Hatfield-Dodds, Z., Lasenby, R., Drain, D., Chen, C., Grosse, R., McCandlish, S., Kaplan, J., Amodei, D., Wattenberg, M., and Olah, C. (2022). Toy models of superposition. Transformer Circuits Thread, Anthropic.

Reproducibility

All experiments were run on Qwen/Qwen2.5-0.5B-Instruct (24 layers, hidden size 896) on CPU in bf16, using the code and recorded outputs in the accompanying repository: https://github.com/ranadhir19/llm-reasoning-demystified

Setup: create an environment with the pinned dependencies in requirements.txt, download the model weights from Hugging Face (Qwen/Qwen2.5-0.5B-Instruct), and set the environment variable QWEN05B_PATH to the local model directory. Then, from the src/ directory:

- Experiment E1 (behavioural): python e1_behavioral.py - writes results/e1_results.json. The run is incremental and resumable; partial results are saved after every prompt. python reparse_e1.py re-extracts the multipliers from the saved generations with the final parser (v3) without re-running the model.
- Experiment E2 (probing): python e2_probing.py - writes results/e2_results.json and caches hidden states in results/e2_hidden_cache.npz.
- Experiment E3 (causal): python e3_causal.py - writes results/e3_results.json. (The patching block of the published results was recomputed with the corrected digit-token scorer: python rerun_e3_patching.py replaces only that block.)
- Experiment E4 (attention and logit lens): python e4_attention_logitlens.py - writes results/e4_results.json.
- Combined runner (single model load, E4 then E2 then E3 then E1): python run_all.py.
- Figures: python make_diagrams.py (F1, F2) and python make_result_figures.py (F3-F6), the latter requiring the results JSON files.
- Validation: python validate.py from the repository root re-checks the package integrity and reports PASS/FAIL per stage.

Raw generations for every E1 prompt, per-layer probe scores, per-prompt ablation log-probabilities, per-layer patching effects, and the full attention and logit-lens records are preserved in the results/ directory of the repository, so every number quoted in Section 8 can be recomputed from the saved data without re-running the model.